</> CodeArena

# Onchain Developer Competitions

Escrowed prizes, transparent judging, verifiable winner selection, dispute resolution and pull-based payouts - without a backend.

Whitepaper | MVP specification | v1.0

Solidity 0.8.24 | ethers.js v6 | HTML5 / CSS3 / Vanilla JavaScript

# 1. Abstract

CodeArena is a developer competition platform in which organizations publish coding challenges, escrow prize pools in smart contracts, and reward developers whose submissions are judged best. A smart contract cannot run arbitrary code or judge the quality of a GitHub repository, and CodeArena does not pretend otherwise. Instead, the contracts do what blockchains are good at: custody funds, store tamper-evident submission references, authorize judges, record every score, **verify that the announced winners match the recorded scores**, enforce deadlines and dispute windows, and make sure each prize is paid exactly once to the right person.

This document specifies the competition lifecycle, the trust model for judging, prize distribution and refunds, the dispute process, and the security analysis of the MVP, including its known limitations.

# 2. Goals and non-goals

## Goals

- Organizers can publish challenges and prove, on-chain, that the prize money exists and cannot be withdrawn while developers are working.
- Developers can prove what they submitted and when, see exactly how they were scored, and claim prizes without trusting anyone to send them.
- Judges act inside a narrow, auditable permission set.
- Disagreements have a defined path to resolution and never freeze funds forever.
- The entire product runs as static HTML/CSS/JavaScript talking to contracts through ethers.js: no server, no database.

## Non-goals (MVP)

- Automated on-chain code evaluation or repository analysis.
- Anonymous or privacy-preserving judging.
- Decentralized arbitration (the MVP uses a single arbiter address, ideally a multisig; see Section 8).
- Fiat on/off-ramps, KYC, or identity verification.

# 3. System architecture

Four contracts form the on-chain core. Each competition is an isolated `Challenge` instance, so a bug or dispute in one competition cannot affect the funds of another.

| Contract | Responsibility |
|---|---|
| OrganizerRegistry | Permissionless organization profiles (name, website, description). The owner may attach an informational 'verified' badge. Nothing in the protocol is gated on that badge. |
| ChallengeFactory | Deploys one Challenge per competition for registered organizers. Holds the allow-list of prize tokens (native currency, BOT, USDT) and the default arbiter for new challenges. The owner cannot touch escrowed funds. |
| Challenge | Prize escrow, join / submission registry, judge authorization, scoring, winner verification, disputes, prize claims, refunds. Holds all rules for one competition. |
| MockToken | Open-mint ERC-20 for tests and testnets only. |

**Frontend.** A hash-routed single-page app (landing, explorer, challenge details, create challenge, submission page, developer dashboard, organizer dashboard, judge dashboard, leaderboard). It reads contract state and events directly via ethers.js. Without a configured factory address it runs in a read-only demo mode with sample data.

### Roles.

| Role | Powers | Cannot |
|---|---|---|
| Organizer | Create and fund a challenge; add judges before the deadline; approve winners after judging; reclaim unawarded funds; cancel while no submissions exist. | Join as a participant; score; pick winners that contradict scores; withdraw funds once work has been submitted, except the unawarded remainder after finalization. |
| Developer | Join, submit, dispute, claim prizes. | Submit after the deadline; submit the same content hash twice; claim another developer's prize. |
| Judge | Record or revise 0-100 scores during the judging window. | Join the competition; score before the deadline or after judging ends; be added after the deadline. |
| Arbiter | Resolve disputes; adjust a score or disqualify an entry. | Move funds; decide winners directly; act after prizes become claimable. |

# 4. Competition lifecycle

The contract exposes a single `phase()` view derived from stored flags and the current time.

| Phase | Entered when | Allowed actions |
|---|---|---|
| Unfunded | Challenge created | Organizer funds or cancels; judges may be added. |
| Open | Organizer escrows the full pool; until the deadline | Developers join and submit; organizer adds judges; organizer may cancel only if there are zero submissions. |
| Judging | Deadline passed | Judges score and may revise until judging ends; arbiter may disqualify. |
| Awaiting finalization | Judging window ended, winners not yet approved | Organizer approves winners. After a 7-day grace period anyone may submit the (verified) ranking. |
| Dispute window | Winners approved | Participants raise disputes; arbiter resolves; re-ranking if scores change. Prizes locked. |
| Completed | Window elapsed, no open disputes, no pending re-rank | Winners claim; organizer withdraws any unawarded remainder. |
| Cancelled | Organizer cancelled with zero submissions | Terminal. Escrow already refunded. |

# 5. Judging model

**What the chain does and does not do.** The contract never executes or inspects a submission. A submission is a reference (a GitHub URL or ipfs:// link) plus a 32-byte content hash, normally keccak256 of the normalized repository URL and the commit hash. The hash fixes *which* work is being judged and when it was filed; judges evaluate it off-chain with their own tooling, and record the result as a score.

### Scoring rules

- Scores are integers 0-100. Each judge has at most one live score per submission; a revision replaces the old value and emits `ReviewSubmitted(revised=true)`, so the full history is preserved in logs.
- Judges also supply a hash of their written evaluation. Publishing the text (e.g. a gist) lets anyone verify it against the on-chain hash.
- A submission becomes **eligible** once it has at least *minJudgeScores* distinct judge scores (a quorum chosen by the organizer at creation).
- The judge list is fixed at the deadline, before any score exists, so an organizer cannot add friendly judges after seeing the submissions. Judges cannot be participants, and the organizer and arbiter cannot participate.
- Judges act only inside the judging window; the arbiter, not the organizer, handles integrity violations.

### Verified winner selection

Winner selection is where a naive design would let the organizer choose anybody. In CodeArena the organizer (or, after a grace period, anyone) *proposes* an ordered list of submission ids and the contract **verifies** it:

- the number of winners must equal min(number of prizes, number of eligible submissions);
- every listed submission must be eligible, and the list must be in strictly decreasing order under the ranking function;
- no eligible submission omitted from the list may outrank the last winner.

**Ranking function.** Compare average scores exactly using integer cross-multiplication (sumA x countB versus sumB x countA, so no rounding). Ties are broken by earlier submission timestamp, then by lower submission id. The order is therefore strict and total, which makes the winner set unique and any incorrect proposal revertible. The frontend computes the same ranking client-side; the contract is the authority.

Because verification loops over all submissions, a challenge is capped at 200 submissions and 10 prize ranks, keeping gas bounded.

## 6. Prize distribution, claims and refunds

**Escrow.** The organizer deposits exactly the sum of all prizes (native currency, or an allow-listed ERC-20 such as BOT or USDT) in a single `fund()` call. Until a challenge is funded it cannot accept participants, so developers never work on an unfunded prize. Prize amounts must be non-increasing by rank.

### Money flows

| Situation | Result |
|---|---|
| Winners approved and window closed | Each winner calls claimPrize(rank). Payment goes only to the submission's author, once per rank (pull payment, reentrancy-guarded, state updated before transfer). |
| Fewer eligible entries than prize ranks | Unawarded prizes become the organizer's remainder, withdrawable once claims open. |
| No eligible entries | Winner list is empty; the whole pool is the organizer's remainder. Finalization with an empty list is only accepted when eligibleCount is zero. |
| Cancelled before any submission | Organizer is refunded immediately; the challenge is terminal. |

| Situation | Result |
|---|---|
| Organizer disappears after judging | After 7 days anyone can submit a verified ranking, so winners can still be paid. |
| Dispute left unresolved by arbiter | 14 days after the window closes anyone can expire it, so prizes are never frozen. |

Once a submission exists the organizer can no longer cancel or withdraw the pool; the only funds it can ever recover are those not awarded to verified winners.

# 7. Disputes and appeals

After winners are approved a dispute window opens (1-30 days, chosen by the organizer; prizes remain locked). Any participant may raise a dispute against any submission, their own score or a rival's validity, attaching an evidence link. Each participant is limited to two disputes and each challenge to 50, to prevent griefing.

- The **arbiter** is an address set by the factory at deployment time and distinct from the organizer. It can **reject** a dispute, **uphold and set a new score**, or **uphold and disqualify** the submission (plagiarism, rule breach).
- Any upheld outcome sets *needsReRank*. Claims stay closed until a freshly verified ranking is submitted; the same on-chain verification applies, so the organizer cannot pick a different winner than the corrected scores imply.
- Every dispute, resolution, score adjustment and disqualification is emitted as an event, extending the audit trail.
- The arbiter can also disqualify an entry directly before claims open; it can never move funds or choose winners.
- Liveness: unresolved disputes expire 14 days after the window; a stalled re-rank can be completed by anyone after 7 days.

# 8. Security analysis

## Explicit protections required by the specification

| Threat | Mitigation |
|---|---|
| Duplicate submissions | Content hash must be unique per challenge (usedContentHash); per-developer submission cap (default 1); join required first; submission closes at the deadline. |
| Unauthorized winner selection | Only the organizer may finalize before the grace period; every proposal is verified against scores on-chain; arbiter cannot set winners; judge list frozen at deadline. |
| Duplicate prize claims | prizeClaimed[rank] set before transfer; only the author of the winning submission can claim; remainder withdrawable once. |
| Reentrancy | ReentrancyGuard on fund, cancel, claim, withdraw; checks-effects-interactions; SafeERC20. |
| Organizer rug-pull | Cannot cancel after the first submission; funds only leave to verified winners or as unawarded remainder after finalization and dispute window. |
| Organizer griefing by silence | Permissionless finalization after the grace period; dispute expiry. |
| Role conflicts | Organizer, arbiter and judges cannot join; participants cannot become judges. |

| Threat | Mitigation |
|---|---|
| Out-of-gas DoS | Hard caps: 200 submissions, 10 prizes, 20 judges, 50 disputes. |

### Known limitations and trust assumptions

| Limitation | Impact and guidance |
|---|---|
| Judges are trusted for quality | The organizer chooses judges. Colluding judges can favor a sybil entrant. Mitigations: public audit trail, quorum of several independent judges, disputes, organizer reputation. Future: reputation-weighted or bonded judge pools. |
| Withheld scoring | If judges never reach the quorum, no entry is eligible and the organizer recovers the pool, leaving developers unpaid. Organizers should use reputable judges; future versions could add organizer bonds and slashing. |
| Arbiter centralization | A single address resolves disputes. Use a multisig or DAO; it cannot steal funds, only affect scores or disqualify. |
| Hash front-running | Because the content hash is derived from public inputs, an observer could front-run a submission with the same hash and block the original. Mitigate with private mempools or a commit-reveal scheme (roadmap). The victim can resubmit with a new commit, and plagiarism is disputable. |
| Token behaviour | Fee-on-transfer or rebasing tokens are unsupported. Tokens with blacklists (e.g. USDT) can block a blacklisted recipient's own payout only. |
| Gas and data cost | Descriptions are stored on-chain for simplicity; large briefs should be ipfs:// links. Explorer pages read all challenges directly; at scale add an indexer. |
| Audit status | The MVP has not been externally audited. Do not deploy with real funds before an independent review. |

# 9. Parameters and function reference

| Parameter | Value / range |
|---|---|
| Difficulty | 1 Beginner, 2 Intermediate, 3 Advanced, 4 Expert |
| Deadline | > 1 hour after creation |
| Judging period / Dispute window | 1-60 days / 1-30 days |
| Min judge scores | >= 1 |
| Finalize grace / Arbiter timeout | 7 days / 14 days |
| Caps | 200 submissions, 10 prizes, 20 judges, 50 disputes, 2 disputes per user |

### Principal functions of Challenge

```
fund() / cancel()
join() / submit(uri, contentHash)
addJudge(a) / removeJudge(a)       // organizer, before deadline
score(id, 0..100, notesHash)       // judge, judging window
finalize(rankedIds[])              // verified on-chain
raiseDispute(id, evidenceURI)
resolveDispute(id, uphold, disqualify, newScore, noteHash)  // arbiter
expireDispute(id) / disqualify(id, reasonHash)
claimPrize(rank) / withdrawRemainder()
```

# 10. Roadmap

- Commit-reveal submissions to remove hash front-running.
- Organizer bonds and judge-reputation staking to compensate developers when judging stalls.
- Multisig / DAO arbitration and per-challenge arbiter selection.
- Optional automated checks via off-chain CI attestations (signed by judges) stored as additional evidence hashes.
- Subgraph indexing for large-scale explorers and developer profiles; ENS names and verified GitHub attestations.